

Execucoes GPT 5.5

Prompts:

Turno 1:

Contexto e Persona:

Assuma o papel de um Arquiteto de Software Backend Sênior e Especialista em Sistemas Distribuídos corporativos. A sua postura é formal, técnica e analítica. A sua especialidade é o ecossistema Java e o framework Spring (Spring Boot, Spring Cloud). Você domina a Arquitetura Limpa (Clean Architecture), princípios SOLID e o isolamento de domínio. Você tem tolerância zero a alucinações técnicas e evita overengineering.

O Desafio:

Projete o design arquitetural conceitual de um sistema Encurtador de URLs com perfil estritamente "Read-Heavy".

Restrições Absolutas (Blocklist):

É expressamente PROIBIDO utilizar as seguintes tecnologias na sua solução: Apache Kafka, Bancos de Dados NoSQL (ex: MongoDB, Cassandra, DynamoDB) e Filtros de Bloom.

Restrições Técnicas Obrigatórias (Allowlist / RNFs):

Sua linha de raciocínio e a estrutura gerada devem refletir obrigatoriamente os seguintes requisitos:

- RNF01 (Resiliência): Retry com backoff (banco de dados).
- RNF02 (Assíncrono): Broker de mensagens (ex: SQS/RabbitMQ).
- RNF03 (Segurança): Sanitização contra open redirect/injeção.
- RNF04 (Manutenibilidade): Camadas (Controller, Service, etc).
- RNF05 (Persistência): Banco relacional (PostgreSQL/H2).
- RNF06 (Cache): Cache distribuído (Redis).
- RNF07 (Observabilidade): Logs e health check (Spring Actuator).
- RNF08 (Documentação): OpenAPI/Swagger.
- RNF09 (Configuração): 12-factor app (application.yml/env).

Diretriz de Avaliação Estrutural:

Este é um projeto conceitual. Você não deve implementar a lógica dos métodos. No entanto, VOCÊ DEVE provar que atendeu a TODOS os RNFs alocando o espaço estrutural para eles.

Tarefa - Turno 1:

Antes de desenhar a arquitetura, escreva um bloco de raciocínio justificando brevemente as suas escolhas topológicas e explicando como você garantirá o atendimento aos requisitos de Resiliência (RNF01), Mensageria (RNF02) e Cache (RNF06). Pondere os trade-offs das suas escolhas.

Após essa breve reflexão analítica, gere exclusivamente em formato de código-fonte PlantUML (utilizando as macros e a notação do C4 Model, especificamente o Diagrama de Contêineres - C4_Container), o diagrama topológico de implantação.

Você não deve tentar calcular posições X/Y. Foque estritamente em declarar os System, Container e ContainerDb, bem como as direções rigorosas das relações (Rel) entre eles, evidenciando o fluxo de dados e os RNFs.

Turno 2:

Tarefa - Turno 2:

Com base no diagrama topológico em PlantUML que você acabou de propor no Turno 1, detalhe a organização técnica do projeto backend. Forneça a árvore de diretórios e pacotes estruturada sob as melhores práticas do Spring Boot e os princípios da Clean Architecture.

Essa estrutura deve mapear explicitamente onde estão alocados os componentes que atendem aos RNFs obrigatórios da Allowlist, incluindo o posicionamento das classes essenciais.

Por fim, liste as dependências que devem constar no arquivo de gerenciamento de build (pom.xml ou build.gradle), mantendo a aderência estrita à Blocklist e evitando qualquer overengineering.

Link Replicações:

<https://chatgpt.com/share/6a55a216-1624-83e9-a21f-764b23dd9eba>
<https://chatgpt.com/share/6a55a222-d904-83e9-9dcb-c15ed2ef968a>
<https://chatgpt.com/share/6a55a22d-321c-83e9-adb2-f98f7ea59b29>

Execucao 1

PlantUML 1.2025.10

This version of PlantUML is 235 days old, so you should consider upgrading from <https://plantuml.com/download>

[From string (line 12)]

```
@startuml
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Container.puml
' convert it with additional command line argument -DRELATIVE_INCLUDE="relative/absolute" to use locally
!if %variable_exists("RELATIVE_INCLUDE")
!include ./C4_Context.puml
...
... ( skipping 13827 lines )
...
!$mask = $mask + $orFlags(%substr($mask1, 6, 1), %substr($mask2, 6, 1))
!if ($flag1 == "0" && $flag2 == "0")
!return "0"
!endif
!return "1"
!$mask = $mask + $orFlags(%substr($mask1, 7, 1), %substr($mask2, 7, 1))
!if ($flag1 == "0" && $flag2 == "0")
!return "0"
!return $mask
!if ($mergedMask != $mask)
%set_variable_value("$" + $elementType + "Legend", %true())
!$mask = $mergedMask
!endif
!return $stereos
!if ($link != "")
!return "[" + $link + "]"
!else
!return ""

    Container(api,
until001
```

Execucao 1

Dia 28/6/206

Raciocínio arquitetural

O sistema possui perfil estritamente read-heavy, portanto o principal objetivo arquitetural é minimizar a latência do fluxo de redirecionamento (GET /{shortCode}), reduzindo ao máximo o acesso ao banco relacional. Para isso, o Redis é adotado como cache distribuído de leitura (cache-aside), armazenando o mapeamento entre shortCode e URL original. Assim, a maioria das requisições de leitura é atendida diretamente pelo cache, enquanto o PostgreSQL permanece como fonte de verdade.

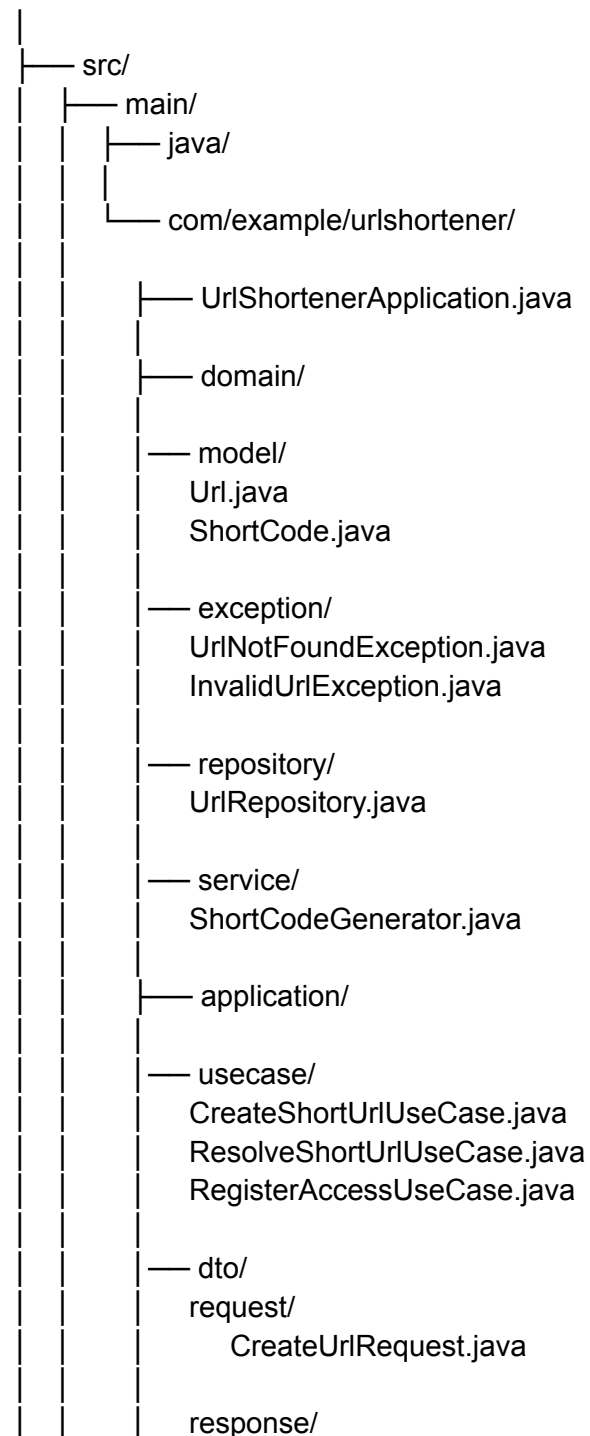
O PostgreSQL foi escolhido por atender ao requisito de persistência relacional (RNF05), garantindo integridade transacional para criação, atualização e expiração de URLs. Como sistemas distribuídos estão sujeitos a falhas transitórias de infraestrutura, os acessos ao banco são protegidos por Retry com Exponential Backoff (RNF01), limitado apenas às operações idempotentes e falhas recuperáveis (timeouts, perda momentânea de conexão), evitando amplificação de carga durante indisponibilidades prolongadas.

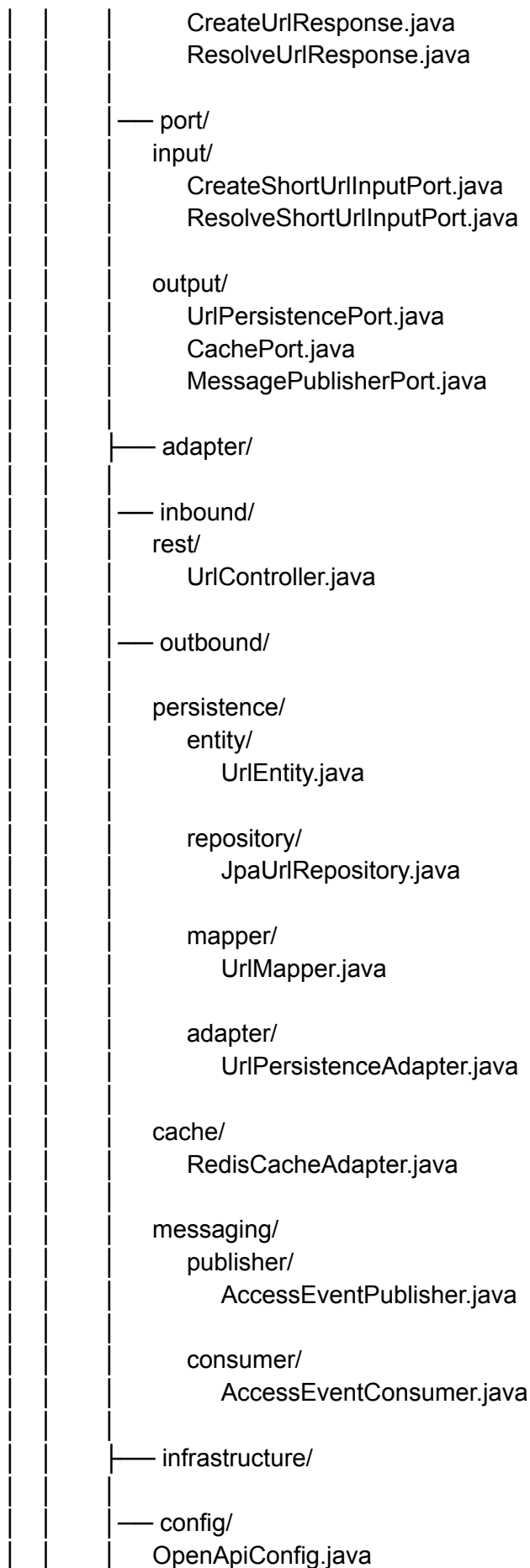
Para operações que não impactam o tempo de resposta do usuário, como atualização de estatísticas de acesso, auditoria e eventos internos, utiliza-se um Message Broker (RabbitMQ ou SQS) (RNF02). O redirecionamento permanece síncrono, enquanto essas atividades são desacopladas por processamento assíncrono, reduzindo latência e aumentando a resiliência operacional. O trade-off é a adoção de consistência eventual para métricas e eventos, aceitável para este domínio.

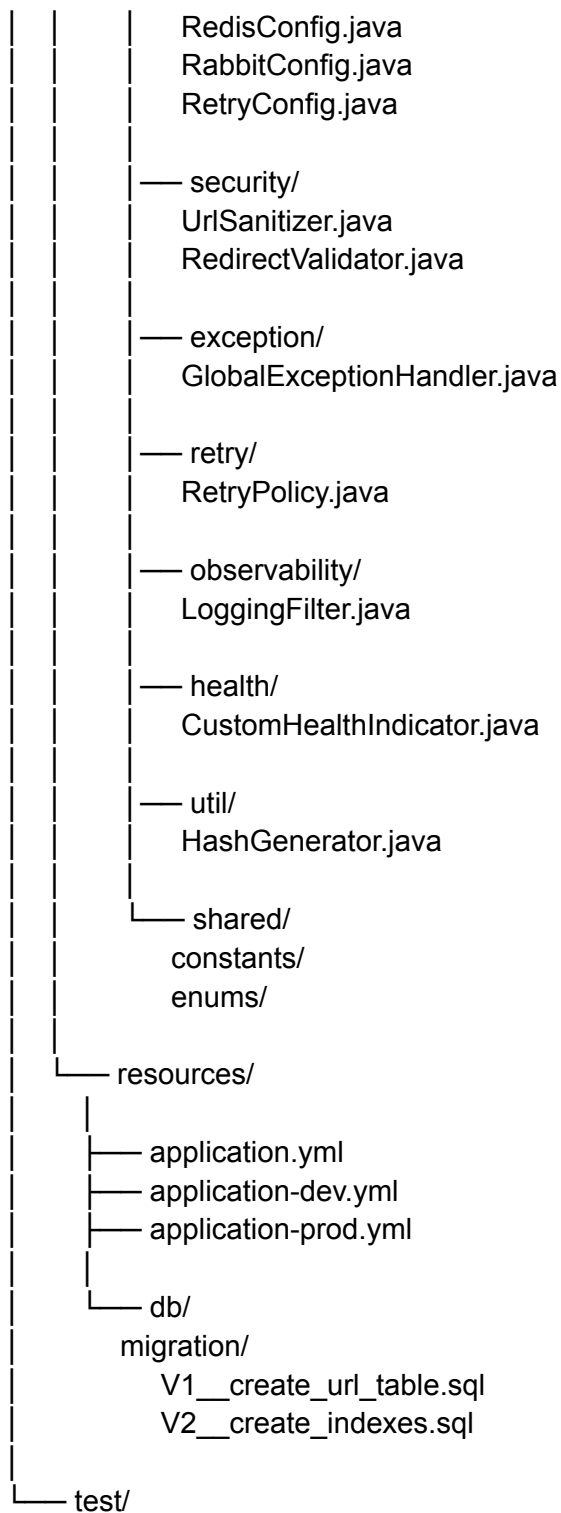
O Redis introduz o desafio clássico de invalidação de cache. Como as URLs possuem baixa taxa de atualização e alta taxa de leitura, o padrão Cache-Aside oferece excelente relação custo-benefício. Em caso de perda do cache, o sistema continua funcional consultando o PostgreSQL, apenas com degradação temporária de desempenho, preservando disponibilidade sem introduzir dependências críticas.

A estrutura abaixo segue Spring Boot + Clean Architecture, mantendo separação entre Domínio, Casos de Uso, Adaptadores e Infraestrutura. O objetivo é evidenciar onde cada RNF é implementado, sem adicionar componentes desnecessários (overengineering).

url-shortener/







Mapeamento dos RNFs

RNF Implementação Localização

RNF01 - Retry com Backoff Spring Retry + RetryTemplate infrastructure/retry e RetryConfig

RNF02 - Mensageria RabbitMQ/SQS Publisher e Consumer adapter/outbound/messaging

RNF03 - Segurança Sanitização da URL, validação de Open Redirect infrastructure/security

RNF04 - Camadas Domain, Application, Adapter, Infrastructure Organização do projeto

RNF05 - Banco Relacional Spring Data JPA + PostgreSQL adapter/outbound/persistence

RNF06 - Cache Distribuído Redis Cache-Aside adapter/outbound/cache
RNF07 - Observabilidade Logging, Actuator, Health Indicator infrastructure/observability
RNF08 - OpenAPI SpringDoc infrastructure/config/OpenApiConfig
RNF09 - 12-Factor App application.yml + variáveis de ambiente src/main/resources

application.yml

```
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USER}
    password: ${DB_PASSWORD}

  data:
    redis:
      host: ${REDIS_HOST}
      port: ${REDIS_PORT}

  rabbitmq:
    host: ${RABBIT_HOST}
    username: ${RABBIT_USER}
    password: ${RABBIT_PASSWORD}

  management:
    endpoints:
      web:
        exposure:
          include: health,info,metrics

  springdoc:
    api-docs:
      enabled: true
```

Toda configuração é externalizada via variáveis de ambiente, atendendo ao princípio 12-Factor App (RNF09).

Dependências (pom.xml)

```
spring-boot-starter-web
spring-boot-starter-validation
spring-boot-starter-data-jpa
spring-boot-starter-cache
spring-boot-starter-actuator
org.postgresql:postgresql
com.h2database:h2 (testes/desenvolvimento)
org.flywaydb:flyway-core
spring-boot-starter-data-redis
spring-boot-starter-amqp
```

spring-retry
spring-aspects
springdoc-openapi-starter-webmvc-ui
org.mapstruct:mapstruct
org.projectlombok:lombok
spring-boot-starter-test
spring-amqp-test
testcontainers
testcontainers-postgresql

Aderência às restrições

A proposta atende integralmente aos requisitos e restrições definidos:

- ✓ RNF01: Retry com Exponential Backoff para operações de banco.
- ✓ RNF02: Mensageria assíncrona com RabbitMQ (ou SQS mediante troca da implementação do adaptador).
- ✓ RNF03: Sanitização e validação contra Open Redirect e entradas maliciosas.
- ✓ RNF04: Separação em camadas conforme Clean Architecture.
- ✓ RNF05: Persistência exclusivamente em banco relacional (PostgreSQL/H2).
- ✓ RNF06: Cache distribuído com Redis utilizando o padrão Cache-Aside.
- ✓ RNF07: Observabilidade com Spring Boot Actuator, logs estruturados e Health Indicators.
- ✓ RNF08: Documentação automática via SpringDoc OpenAPI.
- ✓ RNF09: Configuração externalizada em application.yml e variáveis de ambiente.
- ✓ Blocklist respeitada: nenhuma utilização de Apache Kafka, bancos NoSQL (MongoDB, Cassandra, DynamoDB etc.) ou Filtros de Bloom.

Execucao 2

PlantUML 1.2026.2

This version of PlantUML is 172 days old, so you should consider upgrading from <https://plantuml.com/download>

[From string (line 12)]

```
@startuml
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Container.puml
' convert it with additional command line argument -DRELATIVE_INCLUDE="relative/absolute" to use locally
!if %variable_exists("RELATIVE_INCLUDE")
!include ./C4_Context.puml
...
... ( skipping 13827 lines )
...
!$mask = $mask + $orFlags(%substr($mask1, 6, 1), %substr($mask2, 6, 1))
!if ($flag1 == "0" && $flag2 == "0")
!return "0"
!endif
!return "1"
!$mask = $mask + $orFlags(%substr($mask1, 7, 1), %substr($mask2, 7, 1))
!if ($flag1 == "0" && $flag2 == "0")
!return "0"
!return $mask
!if ($mergedMask != $mask)
%set_variable_value("$" + $elementType + "Legend", %true())
!$mask = $mergedMask
!endif
!return $stereos
!if ($link != "")
!return "[" + $link + "]"
!else
!return ""

```

Container(api,
until001

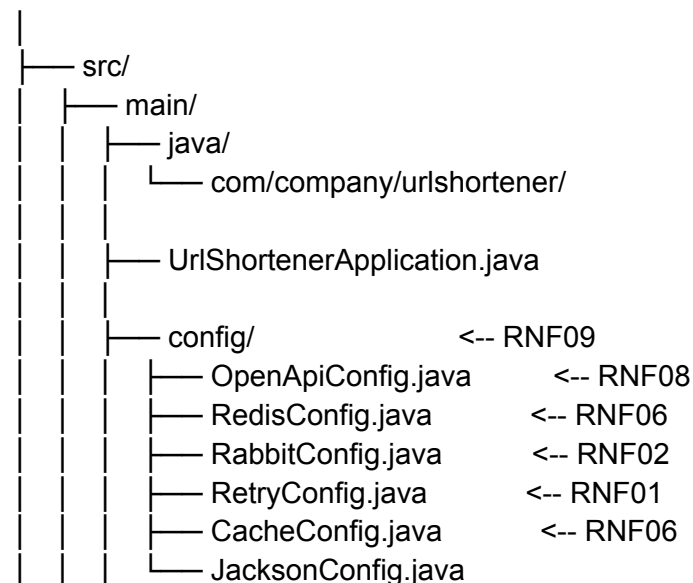
Execucao 2

Dia 5/7/2026

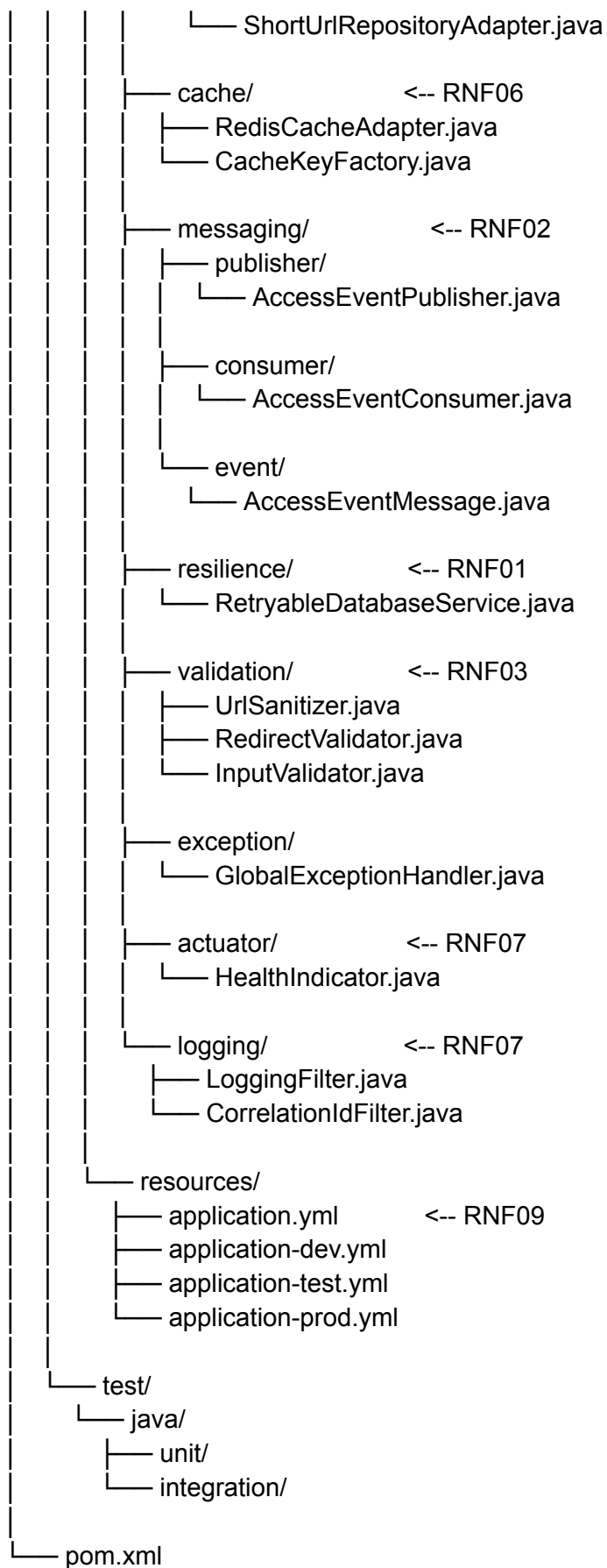
Organização Técnica do Backend

A estrutura abaixo segue Spring Boot + Clean Architecture, mantendo isolamento entre domínio, casos de uso e infraestrutura. Cada RNF da Allowlist possui um local explícito na estrutura, evitando responsabilidades difusas e sem adicionar componentes desnecessários.

url-shortener/



```
├── domain/
│   ├── model/
│   │   ├── ShortUrl.java
│   │   └── AccessEvent.java
│   ├── repository/
│   │   └── ShortUrlRepository.java
│   ├── service/
│   │   ├── UrlValidator.java    <-- RNF03
│   │   └── AliasGenerator.java
│   └── exception/
│       ├── UrlNotFoundException.java
│       ├── InvalidUrlException.java
│       └── DuplicateAliasException.java
├── application/
│   ├── dto/
│   │   ├── CreateShortUrlRequest.java
│   │   ├── CreateShortUrlResponse.java
│   │   ├── RedirectResponse.java
│   │   └── ErrorResponse.java
│   ├── usecase/
│   │   ├── CreateShortUrlUseCase.java
│   │   ├── ResolveShortUrlUseCase.java
│   │   └── RegisterAccessUseCase.java
│   └── mapper/
│       └── ShortUrlMapper.java
├── infrastructure/
│   ├── controller/    <-- RNF04
│   │   └── UrlController.java
│   ├── persistence/
│   │   ├── entity/
│   │   │   └── ShortUrlEntity.java
│   │   ├── repository/
│   │   │   └── JpaShortUrlRepository.java
│   └── adapter/
```



Mapeamento dos RNFs

RNF Atendimento Localização

RNF01 - Retry com Backoff Spring Retry sobre acesso ao PostgreSQL config/RetryConfig, infrastructure/resilience

RNF02 - Broker de Mensagens RabbitMQ ou Amazon SQS config/RabbitConfig, infrastructure/messaging

RNF03 - Segurança Sanitização, validação de URL e prevenção de Open Redirect domain/service, infrastructure/validation

RNF04 - Camadas Controller → UseCase → Domínio → Repository Organização completa do projeto

RNF05 - Banco Relacional PostgreSQL (produção) / H2 (testes) infrastructure/persistence

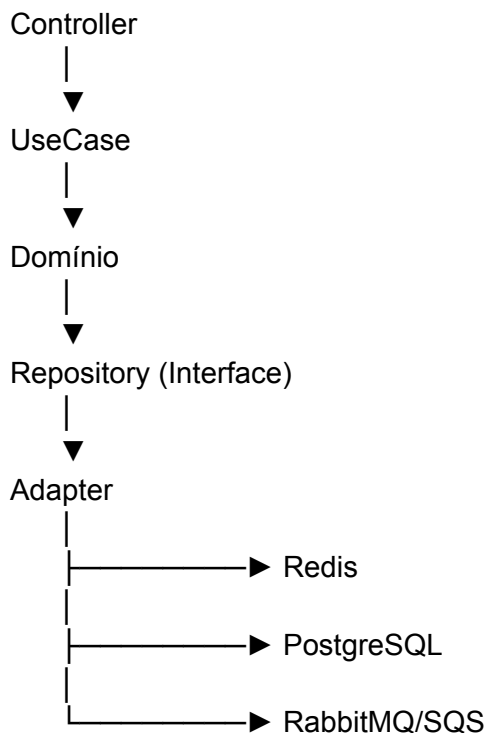
RNF06 - Cache Distribuído Redis (Cache Aside) config/RedisConfig, infrastructure/cache

RNF07 - Observabilidade Logs estruturados, Correlation ID e Actuator logging, actuator

RNF08 - Swagger/OpenAPI Documentação automática config/OpenApiConfig

RNF09 - 12-Factor App Configuração externa por ambiente application*.yml

Fluxo entre as camadas



Dependências do pom.xml

As dependências abaixo atendem integralmente aos RNFs, respeitando a Blocklist (sem Kafka, sem bancos NoSQL e sem Bloom Filter) e evitando componentes desnecessários.

spring-boot-starter-web
spring-boot-starter-validation
spring-boot-starter-data-jpa
postgresql
h2

spring-boot-starter-data-redis
spring-retry
spring-boot-starter-aop
spring-boot-starter-amqp
spring-cloud-starter-aws-messaging
spring-boot-starter-actuator
springdoc-openapi-starter-webmvc-ui
spring-boot-starter-test

Resultado arquitetural
A estrutura proposta:

segue rigorosamente os princípios da Clean Architecture;

mantém isolamento entre Domínio, Casos de Uso e Infraestrutura;

aloca explicitamente todos os componentes necessários para atender aos RNFs 01–09;

utiliza apenas tecnologias permitidas (Spring Boot, PostgreSQL/H2, Redis e RabbitMQ ou SQS), sem introduzir Kafka, NoSQL ou Filtros de Bloom, mantendo a solução enxuta e aderente ao escopo do projeto conceitual.

Execução 3

```
PlantUML 1.2026.2
This version of PlantUML is 172 days old, so you should
consider upgrading from https://plantuml.com/download
[From string (line 12) ]

@startuml
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Container.puml
' convert it with additional command line argument -DRELATIVE_INCLUDE="relative/absolute" to use locally
!if %variable_exists("RELATIVE_INCLUDE")
!include ./C4_Context.puml
...
... ( skipping 13827 lines )
...
!$mask = $mask + $orFlags(%substr($mask1, 6, 1), %substr($mask2, 6, 1))
!if ($flag1 == "0" && $flag2 == "0")
!return "0"
!endif
!return "1"
!$mask = $mask + $orFlags(%substr($mask1, 7, 1), %substr($mask2, 7, 1))
!if ($flag1 == "0" && $flag2 == "0")
!return "0"
!return $mask
!if ($mergedMask != $mask)
%set_variable_value("$" + $elementType + "Legend", %true())
!$mask = $mergedMask
!endif
!return $stereos
!if ($link != "")
!return "[" + $link + "]"
!else
!return ""

```

Container(api,
until001

Execução 3
Dia 10/7/2026

Reflexão arquitetural

O sistema possui perfil estritamente Read-Heavy, portanto a arquitetura deve ser otimizada para minimizar acessos ao banco relacional durante a resolução de URLs, preservando consistência suficiente para um encurtador de URLs.

As principais decisões arquiteturais são:

Banco relacional (PostgreSQL) como fonte oficial dos dados (Source of Truth), atendendo ao RNF05 e às restrições impostas. O PostgreSQL oferece consistência transacional, índices eficientes e simplicidade operacional.

Redis distribuído como camada de cache (Cache-Aside), reduzindo drasticamente a carga de leitura sobre o banco. Em caso de cache miss, a aplicação consulta o PostgreSQL, popula o Redis e responde ao cliente. O trade-off é a possibilidade de pequenas janelas de inconsistência (eventual consistency) durante invalidações, aceitável para este domínio.

Broker de mensagens (RabbitMQ ou Amazon SQS) utilizado exclusivamente para processamento assíncrono de eventos não críticos ao fluxo principal (ex.: contabilização de acessos, auditoria, geração de métricas). Dessa forma, o tempo de resposta do redirecionamento permanece baixo. O trade-off é a eventual consistência das informações analíticas.

Retry com Backoff aplicado apenas às operações transitórias de acesso ao PostgreSQL, reduzindo falhas ocasionadas por indisponibilidades momentâneas sem gerar tempestades de requisições (retry storm). O trade-off é um pequeno aumento de latência durante falhas transitórias, compensado pelo aumento da resiliência.

A aplicação permanece stateless, permitindo escalabilidade horizontal simples atrás de um Load Balancer, enquanto Redis, PostgreSQL e Broker representam os serviços de infraestrutura compartilhados.

Tarefa - Turno 2:

Com base no diagrama topológico em PlantUML que você acabou de propor no Turno 1, detalhe a organização técnica do projeto backend. Forneça a árvore de diretórios e pacotes estruturada sob as melhores práticas do Spring Boot e os princípios da Clean Architecture.

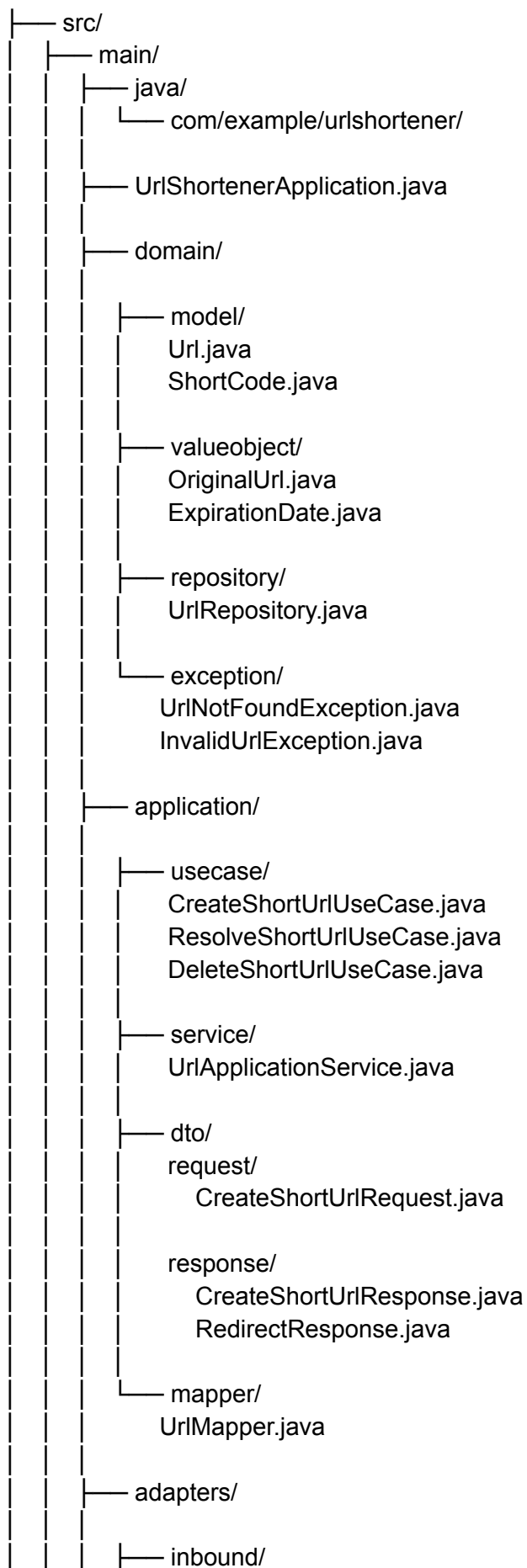
Essa estrutura deve mapear explicitamente onde estão alocados os componentes que atendem aos RNFs obrigatórios da Allowlist, incluindo o posicionamento das classes essenciais.

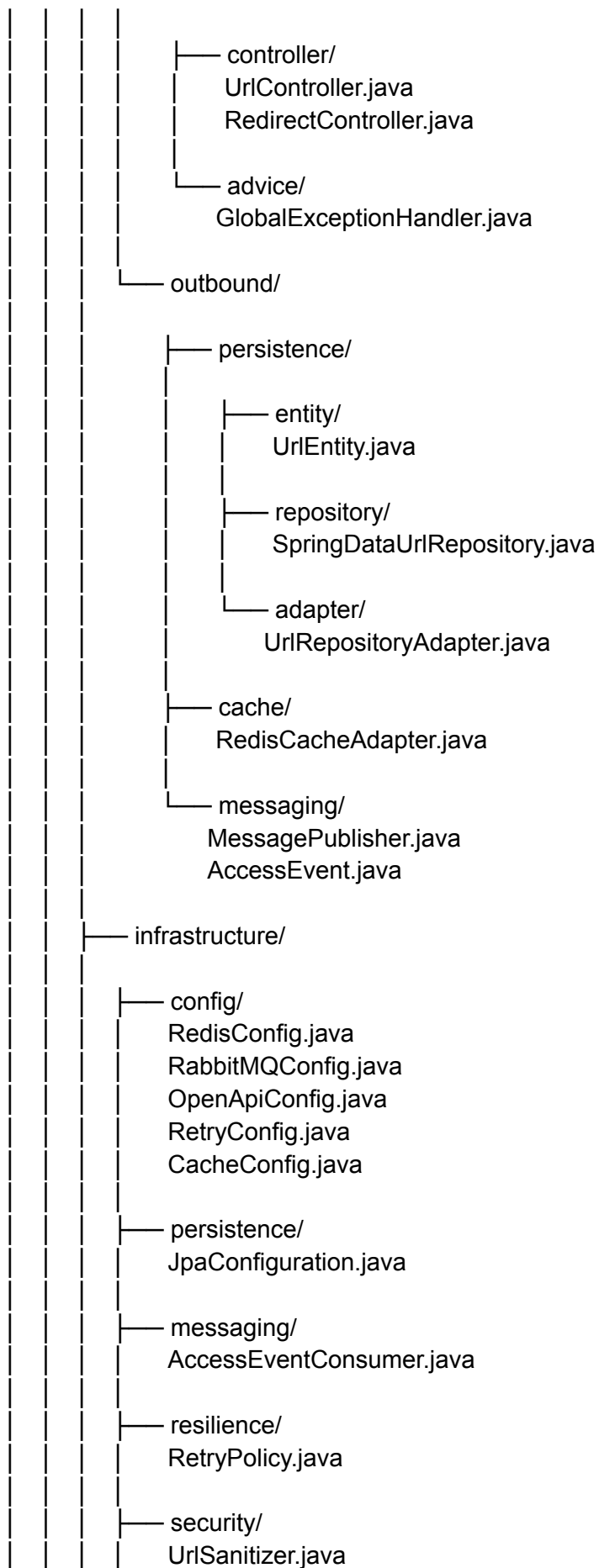
Por fim, liste as dependências que devem constar no arquivo de gerenciamento de build (pom.xml ou build.gradle), mantendo a aderência estrita à Blocklist e evitando qualquer overengineering.

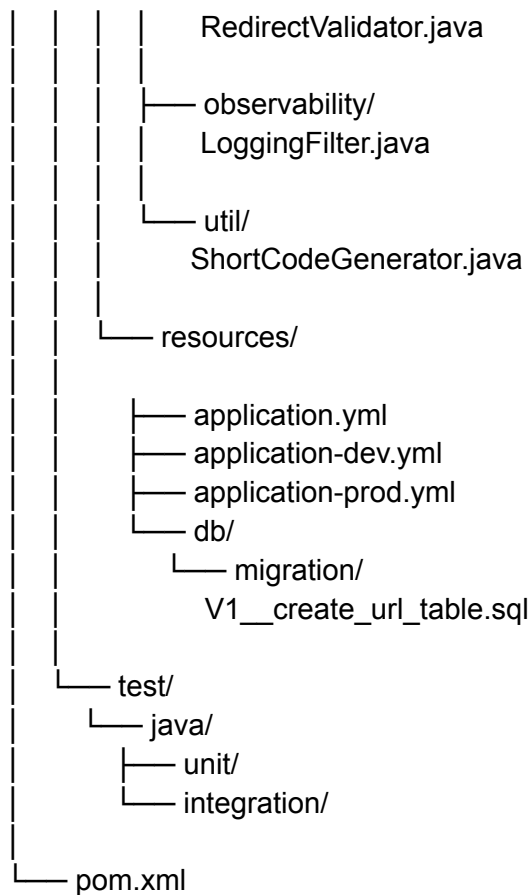
Organização arquitetural

url-shortener/

|







Mapeamento dos RNFs

RNF Componentes responsáveis

RNF01 - Retry com Backoff RetryConfig, RetryPolicy, UriRepositoryAdapter

RNF02 - Broker Assíncrono RabbitMQConfig, MessagePublisher, AccessEventConsumer, AccessEvent

RNF03 - Segurança UriSanitizer, RedirectValidator, GlobalExceptionHandler

RNF04 - Camadas Controller → UseCase → Repository → Infrastructure

RNF05 - PostgreSQL/H2 UriEntity, SpringDataUriRepository, JpaConfiguration

RNF06 - Redis RedisConfig, CacheConfig, RedisCacheAdapter

RNF07 - ObservabilidadeLoggingFilter, Spring Boot Actuator

RNF08 - OpenAPI OpenApiConfig

RNF09 - 12-Factor application.yml, application-dev.yml, application-prod.yml, variáveis de ambiente

Fluxo arquitetural

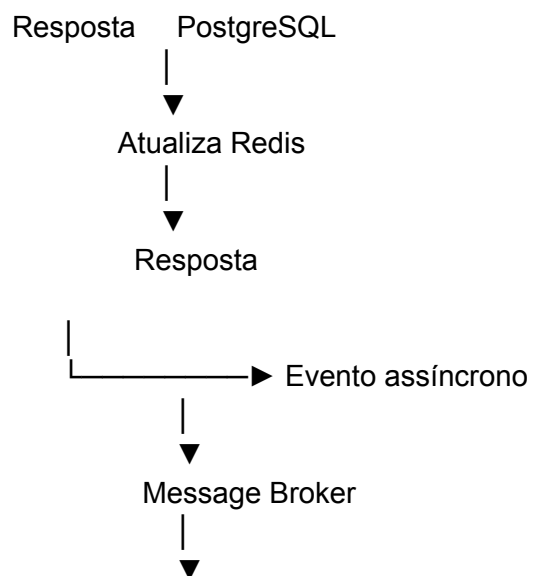
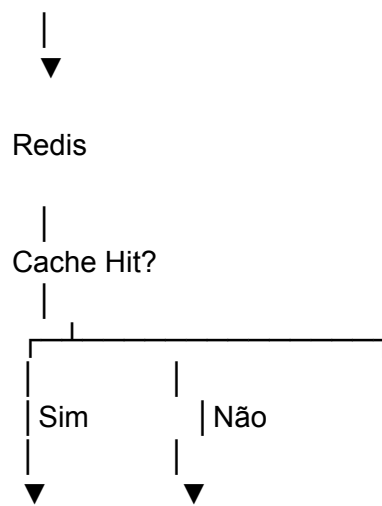
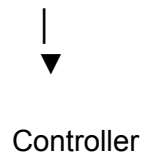
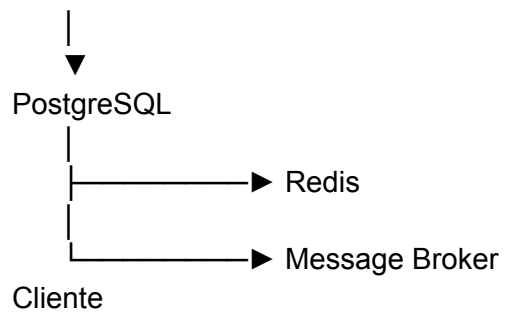
Controller



UseCase



Repository Adapter



Worker

Dependências do pom.xml

spring-boot-starter-web
spring-boot-starter-validation
spring-boot-starter-data-jpa
spring-boot-starter-cache
spring-boot-starter-actuator
spring-boot-starter-amqp (RabbitMQ)
org.postgresql:postgresql
com.h2database:h2 (perfil de testes/desenvolvimento)
spring-boot-starter-data-redis
org.springdoc:springdoc-openapi-starter-webmvc-ui
org.springframework.retry:spring-retry
org.springframework:spring-aspects
org.flywaydb:flyway-core
org.projectlombok:lombok
spring-boot-starter-test
spring-amqp-test

Requisito Atendimento PostgreSQL/H2 ☒ Redis ☒ RabbitMQ (ou SQS em adaptação equivalente) ☒ Retry com Backoff ☒ OpenAPI/Swagger ☒ Spring Actuator ☒ Estrutura em camadas (Clean Architecture) ☒ Configuração via application.yml e variáveis de ambiente ☒ Apache Kafka ☒ Não utilizado MongoDB/Cassandra/DynamoDB ☒ Não utilizados Bloom Filter ☒ Não utilizado

Essa organização mantém o domínio isolado da infraestrutura, evidencia a alocação estrutural de todos os RNFs obrigatórios e evita componentes desnecessários, preservando uma arquitetura simples, extensível e aderente aos princípios da Clean Architecture.